Clean Beauty Recommender: PySpark Edition

Sheriann McLarty – DATA 612
Date: July 19, 2025

Project Overview

This project aimed to build a collaborative filtering recommender system using PySpark's ALS algorithm to suggest clean beauty products. The dataset contained over 10,000 user reviews, product features, and demographic traits such as skin tone and eye color.

But what started as a simple "let's run ALS" journey became a rollercoaster across different platforms—each with its own quirks, limits, and error messages.

Data Cleaning: More than Skin Deep

- My data was **originally clean in Python/pandas**, but Spark? She said *no ma'am*.
- Every column had to be cast and recast to survive Spark's **strict schema enforcement**. `rating_x` was string. `price_usd_x` was string. Even fields like `author_id` needed a full `StringIndexer` makeover.
- I hit roadblock after roadblock—**CAST_INVALID_INPUT**, **Py4JSecurityException**, and missing winutils.exe warnings on PyCharm made debugging harder than blackhead extraction without steam.

The Tool Troubles

Databricks

- **Free edition** was cute until it started telling me I used up all my quota.
- No R, no cluster scaling, no escape from Spark SQL constraints.

PyCharm

- I pivoted to **local development**. Set up a Spark 3.1.0 env with Java 8, manually edited PATH variables, and launched via PyCharm.
- **Winutils.exe? Missing.** Java memory errors? Constant.
  Still, it gave me the control Databricks wouldn't.

Result?

PySpark + PyCharm became my ride-or-die setup. I finally got ALS to run, and Spark stopped screaming at my data types.

ALS Model Summary

- **Train/Test Split:** 80/20
- **RMSE:** 1.7011

- **Total Ratings:** 10,897

```
| user        | item   | rating |
|-------------|--------|--------|
| 1342721722  | 288.0  | 4.0    |
| 1026741323  | 288.0  | 4.0    |
| 1073381846  | 288.0  | 2.0    |
```

Why Spark?

While this dataset could've lived happily in pandas or base Python, Spark gave me: - Speed in processing larger files - A taste of **scalable production-level recommender systems** - A reality check on how annoying distributed systems can be when they're not fully supported (Databricks free tier)

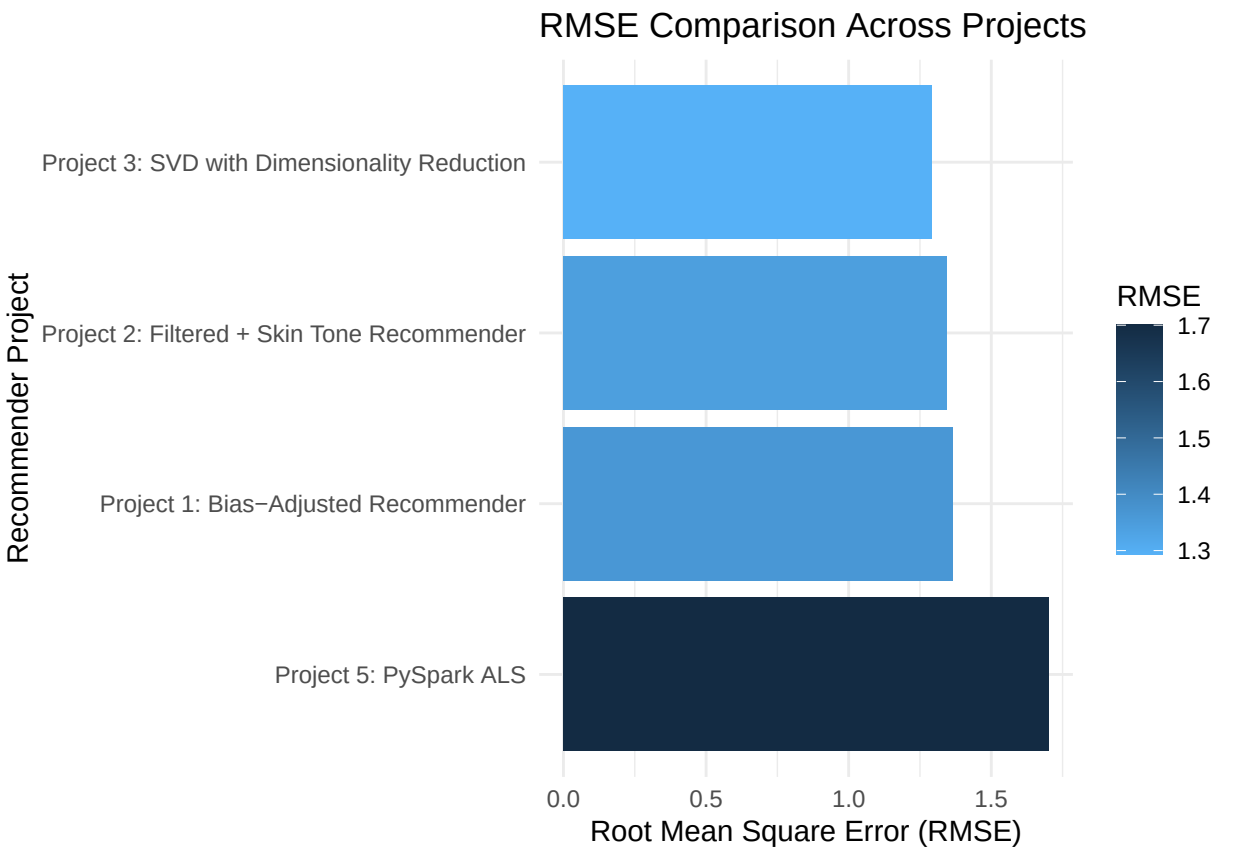
```
rmse_df <- data.frame(
  Project = c(
    "Project 1: Bias-Adjusted Recommender",
    "Project 2: Filtered + Skin Tone Recommender",
    "Project 3: SVD with Dimensionality Reduction",
    "Project 5: PySpark ALS"
  ),
  RMSE = c(1.3652, 1.3446, 1.2936, 1.7011)
)

rmse_df
```

```
##                                       Project   RMSE
## 1         Project 1: Bias-Adjusted Recommender 1.3652
## 2  Project 2: Filtered + Skin Tone Recommender 1.3446
## 3 Project 3: SVD with Dimensionality Reduction 1.2936
## 4                      Project 5: PySpark ALS 1.7011
```

```
library(ggplot2)

ggplot(rmse_df, aes(x = reorder(Project, -RMSE), y = RMSE, fill = RMSE)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  scale_fill_gradient(low = "#56B1F7", high = "#132B43") +
  labs(title = "RMSE Comparison Across Projects",
       x = "Recommender Project",
       y = "Root Mean Square Error (RMSE)") +
  theme_minimal()
```



Final Thoughts

What should've taken two days took a week. I switched platforms, fought Spark errors, and Googled like my life depended on it. But in the end—I built it. I learned the value of **persistence, platform flexibility**, and **reading the schema output before casting**.

And now I have a working recommender. So, cheers to that.

Future Takeaways

- **Know your tool limits early**: Free-tier Databricks has restrictions that can block progress fast.
- **Check schema outputs first**: Data types silently break pipelines.
- **Have a local backup plan**: PyCharm with the right Java/Hadoop config saved this project.
- **Be flexible with your environment**: Sometimes the fastest solution isn't the cloud—it's you with a terminal, a debugger, and sheer determination.